NUVEXI

Volunteer Coordination Platform

Technical Document

Version 1.0 | 26 October 2026

Prepared by

Group C262W-5103

Tayyab Rashid	a1988298
Neev Aryan	a3192763
Nashan Dsouza	a1867098
Nguyen Duc Tung Bui	a1976012
Abid Al Mamun	a3181228

Document Control

Version	Date	Author	Change
1.0	26 October 2026	Group C262W-5103	First issue

Review & Approval

Technical Document Approval History

Approving Party	Version Approved	Signature	Date
Tayyab Rashid	1.0		26 October 2026
Neev Aryan	1.0		26 October 2026

Technical Document Review History

Reviewing Party	Version Reviewed	Signature	Date
Nashan Dsouza	1.0		26 October 2026
Nguyen Duc Tung Bui	1.0		26 October 2026

Contents

1. Introduction 5
1.1 Purpose and Scope 5
1.2 Audience 5
1.3 System Summary 5
1.4 Definitions and Abbreviations 5
2. Technology Selection and Justification 6
2.1 Backend Stack 6
2.2 Frontend Stack 6
2.3 Development and Test Tooling 7
2.4 Summary of Choices 7
3. System Architecture 8
3.1 Overview 8
3.2 Backend Architecture 8
3.3 Frontend Architecture 9
3.4 Request Flow 9
4. Data Model 11
4.1 Conventions 11
4.2 Entity Relationship Diagram 12
4.3 Entities by Domain 13
4.4 Status Lifecycles 14
4.5 Database Constraints and Schema Management 14
5. API Design 15
5.1 Conventions 15
5.2 Resource Overview 15
5.3 Contract Between Frontend and Backend 15
6. Security and Privacy 17
6.1 Authentication and Sessions 17
6.2 Authorisation and Role Matrix 17
6.3 CSRF, CORS and Cookies 18
6.4 Rate Limiting and Account Enumeration 18
6.5 Activity Log 18
6.6 Data Retention and De-identification 18
7. Key Workflows 19

8. Frontend Implementation Notes 21
9. Quality Assurance 22
10. Local Development Environment 23
11. Known Limitations 24
12. References 25
Appendix A. Endpoint Catalogue 27
Appendix B. Environment Variables 30
Appendix C. Error Codes 31

1. Introduction

1.1 Purpose and Scope

This document describes how the Nuvexi volunteer coordination platform is built. It covers the technology choices and the reasons for them, the architecture of the backend and frontend, the data model, the API, the security design, and how the system is tested and run locally. Project background, requirements gathering and evaluation are covered in the project report and are not repeated here.

1.2 Audience

The document is written for developers and technical staff who will maintain or extend the platform. It assumes a working knowledge of Java, Spring, TypeScript and React.

1.3 System Summary

Nuvexi lets an organisation publish volunteer events, collect applications through custom forms, place accepted volunteers on shifts and record their attendance. It has three roles:

- Volunteer: keeps a profile, browses published events, applies, and sees their shifts and hours.
- Coordinator: creates and runs events, builds application forms, reviews applications, allocates shifts and records attendance.
- Administrator: manages user accounts and platform settings, reads the activity log, and can manage any event.

The system is split into two codebases: a Spring Boot REST API (the backend) and a React single-page application (the frontend). They are kept in separate repositories and communicate only over HTTP.

1.4 Definitions and Abbreviations

Term	Meaning
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
DTO	Data Transfer Object, a class that defines a request or response body
JPA	Jakarta Persistence API
SPA	Single-Page Application
Public ID	The random UUID that identifies a record outside the server
Outbox	A database table of emails waiting to be sent

2. Technology Selection and Justification

2.1 Backend Stack

Java 21 is the current long-term support release of Java [1]. It gives records, pattern matching and switch expressions, which the code uses for DTOs and status rules, and it will receive security updates for several years.

Spring Boot 4.1 [2] is the application framework. It provides the web layer, dependency injection, configuration binding, scheduling and an embedded server, so the application runs as a single jar. Most problems the platform has (sessions, validation, mail, transactions) have a standard Spring answer, which keeps the amount of custom infrastructure code small.

Spring Security [3] handles authentication, the session, CSRF protection and CORS. **Spring Session JDBC** [4] stores HTTP sessions in PostgreSQL instead of server memory. This was chosen so that the server can end every session for an account, for example after a password change or when an administrator disables the account.

Spring Data JPA with Hibernate [5] maps the domain classes to tables. Repositories cover most queries without hand-written SQL, and optimistic locking is available through a single `@Version` field.

PostgreSQL 17 [6] is the database. It is open source, has strong transactional guarantees, and supports the partial and functional indexes, JSON columns and triggers that the data model relies on (Section 4.5).

springdoc-openapi [7] produces an OpenAPI 3 description of the API [8] and a Swagger UI directly from the controllers. The frontend generates its TypeScript types from this description.

Lombok removes getter, constructor and logger boilerplate from entities and services.

2.2 Frontend Stack

React 19 [9] with **TypeScript** is used for the user interface. React's component model suits a screen-heavy application with many shared pieces such as tables, dialogs and form fields. TypeScript catches contract mistakes between the frontend and the API at compile time.

Vite [10] is the build tool and development server. It starts quickly and reloads changes almost instantly, and its production build needs no extra configuration.

React Router handles client-side routing, including the role guard on each route.

TanStack Query [11] manages server state: fetching, caching, refetching and invalidation after a change. This removes the need for a separate global store, because almost all shared state in the application comes from the server.

React Hook Form [12] and **Zod** [13] handle forms. Zod schemas describe each form once, and React Hook Form uses them for validation without re-rendering the whole form on each keystroke.

Tailwind CSS 4 [14] is used for styling. The colour palette, fonts and spacing are defined once as design tokens and used everywhere, which keeps the look consistent and makes contrast checks simple.

openapi-typescript [15] generates TypeScript types from the backend's OpenAPI description, so request and response shapes are not written by hand.

2.3 Development and Test Tooling

Tool	Purpose
Maven wrapper	Builds the backend with a pinned Maven version, no global install needed
Docker Compose	Runs PostgreSQL and the mail catcher locally
Mailpit [16]	Catches all outgoing email during development
JUnit 6, Spring Test	Backend unit and integration tests
Vitest, Testing Library	Frontend unit and component tests
Playwright [17]	End-to-end browser tests
ESLint, Prettier	Frontend linting and formatting

2.4 Summary of Choices

Area	Chosen	Main reason	Alternative considered
Backend language	Java 21	Long-term support, strong typing, mature ecosystem	Node.js with TypeScript
Backend framework	Spring Boot	Security, sessions, mail and scheduling built in	Express or NestJS
Database	PostgreSQL	Partial indexes, JSON columns, triggers	MySQL
Data access	Spring Data JPA	Less query code, optimistic locking	jOOQ or plain JDBC
Session model	Server sessions in PostgreSQL	Sessions can be revoked at once	Stateless JWT
API description	OpenAPI via springdoc	Generated from code, drives client types	Hand-written specification
UI library	React	Component reuse, large ecosystem	Angular or Vue
Build tool	Vite	Fast feedback, simple configuration	webpack
Server state	TanStack Query	Caching and invalidation without a global store	Redux Toolkit
Forms	React Hook Form with Zod	One schema for types and validation	Formik with Yup
Styling	Tailwind CSS	Shared tokens, no separate stylesheet per component	CSS Modules
End-to-end tests	Playwright	Multiple browsers, reliable waiting	Cypress

JWT was the main alternative weighed for sessions. A signed token cannot be withdrawn before it expires without adding a deny list, which brings back server state anyway. Because the platform must sign a user out everywhere when their password changes or their account is disabled, server-side sessions were the simpler option.

3. System Architecture

3.1 Overview

The platform follows a client and server design (Figure 1). The browser loads the React application, which calls the REST API with JSON. The API is the only component that talks to the database and the mail server. Authentication uses an HTTP-only session cookie, and every state-changing request also carries a CSRF token header.

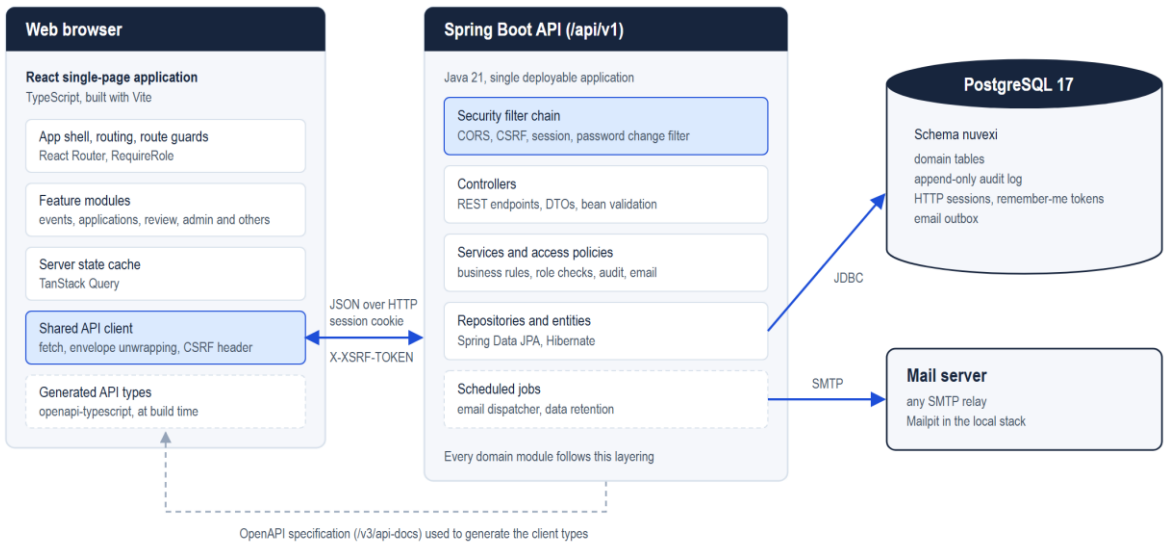


Figure 1. System architecture

The OpenAPI description published by the backend is the contract between the two codebases. When an endpoint changes, the frontend types are regenerated from it, and any mismatch shows up as a type error.

3.2 Backend Architecture

The backend is a single Spring Boot application split into modules by business domain. Each module owns its entities, repositories, services and controllers.

Module	Responsibility
iam	Accounts, registration, sign-in, passwords, roles, invitations
profile	Volunteer profiles, qualifications, degree documents
event	Events, event roles, shifts, co-managers
form	Application forms and their questions
application	Volunteer applications, answers, eligibility, review
allocation	Placing volunteers on shifts
attendance	Attendance records and hours
audit	The append-only activity log
notification	Email outbox and dispatcher

Module	Responsibility
retention	Removal of personal data after the retention period
privacy	The published privacy notice
settings	Platform settings that administrators change at runtime
common	Base entity, response envelope, errors, logging, security handlers

Inside a module the code is layered:

- Controllers accept requests, validate DTOs with Bean Validation and return the standard response envelope. Business rules live in the services.
- Services carry the business rules and own the transaction boundary.
- Access policies (for example EventAccessPolicy and UserAccessPolicy) answer one question each, such as "may this user manage this event". Services call them before reading or changing a record, so most authorisation rules sit in one place per domain instead of being spread across controllers.
- Repositories are Spring Data interfaces. Queries that need locking or projections are declared on the repository.
- Domain classes are JPA entities. State changes go through methods on the entity, such as `withdraw()` or `publish()`. The service checks that the transition is allowed before calling them.

Cross-module work, such as sending an email when an application is accepted, is mostly done by calling the other module's service. Where a service needs a quick check against another module's data, for example counting a volunteer's open shifts before a role change, it uses that module's repository.

3.3 Frontend Architecture

The frontend source is organised by feature:

- src/app holds the application shell, navigation, the route table, the role map for routes and the home pages for each role.
- `src/features` holds one folder per area: auth, profile, opportunities, applications, activities, events, form-builder, forms, review, allocation, attendance, requirements and admin. A feature folder uses up to four subfolders as needed: `api` (calls and query keys), `model` (schemas and pure logic), `components` and `routes` (pages).
- src/shared holds code used across features: the API client, the generated API types, session handling, UI primitives such as buttons, dialogs and pagination, and the date and time helpers.

Features mostly import from `shared` rather than from each other. A few components, such as the form message and the question field, are reused directly from the feature that owns them.

Routes are declared in one table. Every signed-in route is wrapped in a `RequireRole` guard that reads its allowed roles from `routeAccess.ts`. The guard only decides what the user is shown. The server makes the real access decision on every request.

3.4 Request Flow

The following steps trace a volunteer submitting an application:

1. The volunteer fills in the event's form. The answers are checked in the browser against each question's type and rules before anything is sent.

2. The submit handler calls api('/applications', { method: 'POST', body }). The shared client adds the X-XSRF-TOKEN header from the CSRF cookie and sends the session cookie.
3. Spring Security checks the CSRF token, loads the session from PostgreSQL and confirms the account is not waiting on a password change.
4. ApplicationController validates the DTO and calls ApplicationService.
5. The service checks eligibility (age, required checks, expertise and degree), validates each answer against the current question version, and saves the application in one transaction. The caller must also be a volunteer with a verified email and a complete profile.
6. The audit service records the action in its own transaction.
7. The controller returns { success: true, data: ... }. The frontend unwraps the envelope and opens the volunteer's application list, which shows a confirmation.

4. Data Model

4.1 Conventions

All entities except `LoginAttempt` and `AuditEvent`, which are simple append-only records, extend `BaseEntity`. It gives them:

- id, an identity primary key that never leaves the server;
- public_id, a random UUID used in URLs and responses, so record numbers cannot be guessed or counted;
- created_at and updated_at, set automatically.

Other conventions:

- Tables live in the nuvexi schema.
- Entities that users edit concurrently carry a version column for optimistic locking. A conflicting update returns 409 STALE_RESOURCE, and the frontend asks the user to reload.
- Enums are stored as strings, so adding a value does not renumber existing rows.
- Flexible structures, such as question validation rules and multiple-choice answers, are stored as JSON columns.
- Records are rarely deleted. Shifts, allocations and form questions are marked with a removed_at or retired_at timestamp instead, so history stays intact.
- Fields holding personal data are marked with a @PersonalData annotation, which the activity log uses to keep their values out of its entries.

4.2 Entity Relationship Diagram

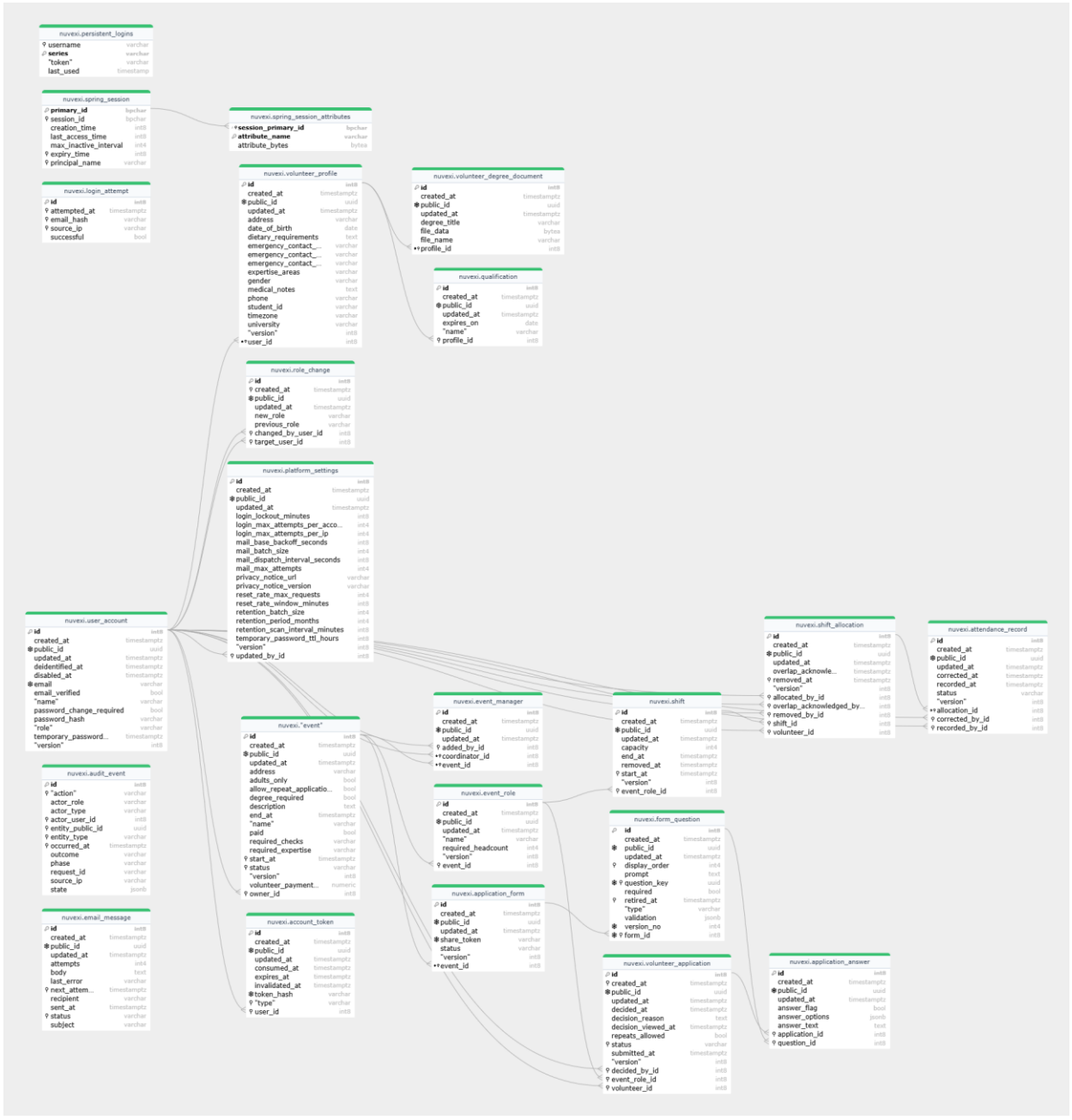


Figure 2. Entity relationship diagram

4.3 Entities by Domain

Domain	Entity	Key fields and relationships
Identity	UserAccount	name, email (unique, lower case), password hash, role, email verified, disabled at, password change required
Identity	AccountToken	user, type (email verification or password reset), token hash, expiry, consumed at
Identity	RoleChange	target user, changed by, previous role, new role
Identity	LoginAttempt	email hash, source IP, time, success flag
Profile	VolunteerProfile	one per user; contact details, emergency contact, date of birth, dietary and medical notes, expertise areas
Profile	Qualification	profile, name, expiry date
Profile	DegreeDocument	profile, degree title, PDF file (binary)
Event	Event	name, address, start and end, status, eligibility rules, payment, owner
Event	EventManager	event, coordinator, added by
Event	EventRole	event, name, required headcount
Event	Shift	event role, start and end, capacity
Form	ApplicationForm	one per event, status, share token
Form	FormQuestion	form, question key, version number, type, prompt, required flag, validation (JSON)
Application	VolunteerApplication	volunteer, event role, status, decision details
Application	ApplicationAnswer	application, question version, answer text, options or flag
Allocation	ShiftAllocation	shift, volunteer, allocated by, overlap acknowledgement
Attendance	AttendanceRecord	one per allocation, status, recorded by, corrected by
Audit	AuditEvent	time, action, phase, outcome, entity, actor, request ID, changed fields (JSON)
Notification	EmailMessage	recipient, subject, body, status, attempts, next attempt time
Settings	PlatformSettings	single row of runtime settings

Three further tables are not mapped by JPA. Spring Session creates `spring_session` and `spring_session_attributes` for HTTP sessions, and `data.sql` creates `persistent_logins` for remember-me tokens.

4.4 Status Lifecycles

The event and application lifecycles are enforced by a `canTransitionTo` method on their status enums, which the services call before changing anything. The other statuses have simple rules that the services check directly.

Entity	Allowed transitions
Event	DRAFT to PUBLISHED or CANCELLED; PUBLISHED to CANCELLED or COMPLETED
Application form	DRAFT to PUBLISHED
Application	SUBMITTED to WITHDRAWN, ACCEPTED or REJECTED
Attendance	PRESENT, LATE or ABSENT; can be corrected; PRESENT and LATE count towards hours
Email	QUEUED to SENT, or to ABANDONED after the last failed attempt

4.5 Database Constraints and Schema Management

Some rules cannot be expressed through JPA annotations and are created by `data.sql`, which runs on every start and is written to be safe to repeat:

- A partial unique index allows only one application per volunteer per event role, unless the event allows repeat applications. Withdrawn applications do not count, so a volunteer who withdraws can apply again.
- A partial unique index allows only one live allocation per volunteer per shift.
- Functional indexes on lower(name) and lower(address) support case-insensitive search.
- Triggers on audit_event reject any UPDATE, DELETE or TRUNCATE, so the activity log cannot be altered, even by the application.

The tables themselves are created and updated by Hibernate (`ddl-auto=update`). This kept the schema in step with the code during development, but it cannot drop or rename columns safely (see Section 11).

5. API Design

5.1 Conventions

- All endpoints sit under /api/v1. Public endpoints sit under /api/v1/public or /api/v1/auth.
- Resources are nouns and use standard HTTP methods. PATCH is used for state changes such as withdrawing an application or changing an event's status.
- Every response uses the same envelope:

```
{ "success": true, "data": { ... }, "error": null, "timestamp": "2026-10-26T01:00:00Z" }
```

- On failure, error carries a machine-readable code, a message that is safe to display, optional field errors, and the request ID. Error bodies never contain exception text. The codes are listed in Appendix C.
- List endpoints accept page, size and sort. The default page size is 20 and the maximum is 100. Sorting is only allowed on a fixed list of fields per endpoint.
- Every request is given a request ID, or keeps the one sent in an `X-Request-Id` header. The ID appears in each log line for that request and in every error body, so a user's error report can be matched to the server log.

5.2 Resource Overview

Resource	Purpose
/auth	Register, sign in and out, verify email, reset and change password, CSRF token
/users	Account administration (administrators only)
/profiles	Volunteer profile, qualifications and degree document
/events	Events, event roles, co-managers and ownership
/events/{id}/shifts, /shifts	Shift scheduling
/events/{id}/form	Application form builder
/public/forms	Read a published form through its share link
/applications	Submit, withdraw and review applications
/allocations	Place and remove volunteers on shifts
/attendance	Record attendance and view hours
/audit-events	Activity log (administrators only)
/admin/settings	Platform settings (administrators only)
/public/privacy-notice	Current privacy notice

The full list of endpoints is in Appendix A. While the backend is running, Swagger UI at `/api/swagger-ui.html` lists every endpoint with its request and response schema.

5.3 Contract Between Frontend and Backend

The frontend never defines API shapes by hand. Running `npm run api:types` against a running backend regenerates `src/shared/api/generated/schema.d.ts`, and `shared/api/types.ts` re-exports the types the

features use. A backend change that renames or removes a field therefore breaks the frontend type check instead of failing silently at runtime.

6. Security and Privacy

6.1 Authentication and Sessions

- Passwords must be at least 12 characters and are hashed with bcrypt through Spring's delegating encoder, which records the algorithm with each hash so it can be upgraded later [18].
- After sign-in the session ID is changed to prevent session fixation. Sessions are stored in PostgreSQL and expire after 30 minutes of inactivity.
- The session cookie (NUVEXISESSION) is HTTP-only, SameSite=Lax, and Secure outside local development [19].
- Volunteers can choose "remember me", which issues a persistent token valid for up to seven days. Coordinators and administrators cannot, because their accounts hold more access.
- Email verification and password reset links carry random tokens. Only a SHA-256 hash of each token is stored. Verification links last 24 hours and reset links 30 minutes, and each token works once.
- Accounts created by an administrator receive a temporary password by email. Until it is replaced, `PasswordChangeRequiredFilter` refuses every request with `403 PASSWORD_CHANGE_REQUIRED`, except the password change itself, signing out, reading the current account and the public endpoints.
- Resetting a password, disabling an account or an administrator choosing "sign out everywhere" deletes all of the account's sessions and remember-me tokens. When users change their own password, every other session and all remember-me tokens are deleted, and the session they are using is kept.

6.2 Authorisation and Role Matrix

Capability	Volunteer	Coordinator	Administrator
Edit own profile, apply to events	Yes	No	No
Browse published events	Yes	No	Yes
Create events	No	Yes	Yes
Manage an event, its form, shifts and applications	No	Owner or co-manager	Any event
Add or remove co-managers	No	Owner	Any event
Invite a new coordinator to an event	No	No	Yes
Record attendance	No	Owner or co-manager	Any event
Manage user accounts and roles	No	No	Yes
Change platform settings, read activity log	No	No	Yes

The frontend hides links and routes a role cannot use, but every rule above is enforced again by the backend. An owner or co-manager who is demoted to volunteer loses access to the event at once. A refused request returns `403` and is written to the activity log.

6.3 CSRF, CORS and Cookies

Because authentication relies on a cookie, the API uses Spring Security's CSRF protection for single-page applications [20]. The server sets an `XSRF-TOKEN` cookie that the frontend can read, and the frontend copies it into the `X-XSRF-TOKEN` header on every request that is not `GET`. A third-party site can make the browser send the cookie but cannot read it, so it cannot produce the header.

CORS only allows the origins listed in `CORS_ALLOWED_ORIGINS`. A wildcard is not used, since together with credentials it would let any site call the API as the signed-in user.

6.4 Rate Limiting and Account Enumeration

Failed sign-ins are recorded against a hash of the email address and against the source IP. By default an account is refused sign-in while it has five failures in the last 15 minutes, and an IP address while it has twenty. Password reset requests are limited to three per hour per account. These limits are platform settings and can be changed by an administrator.

Sign-in and password reset do not reveal whether an email address is registered. A wrong password and an unknown address return the same error, and a reset request gets the same response either way. Registration with an address that is already in use returns a conflict with a generic message.

6.5 Activity Log

Security-relevant and business actions are written to `audit_event`, for example sign-ins, role changes, application decisions and setting changes. Most changes are recorded as a PRE entry before the work and a POST entry after it, so an action that started but did not finish can be seen. Simple actions, such as changing settings or inviting a user, write a single POST entry. Entries are written in a separate transaction so that a refused or failed action is still recorded.

For fields marked `@PersonalData`, the log stores the field name only, never the old or new value. The table is append-only at database level (Section 4.5), and administrators read it through a filtered, paged screen.

6.6 Data Retention and De-identification

Volunteer personal data is kept for a set period after the end of the last shift the volunteer was allocated to. The period is at most 12 months, which is also the default. A scheduled job checks once a day for volunteers past that point and de-identifies them one at a time, each in its own transaction. Profile details, qualifications, degree documents, application answers, decision reasons and emails are erased. The account keeps a random password hash that cannot be matched. Application, allocation and attendance rows remain, so event totals stay correct without identifying anyone. This follows the Australian Privacy Principles on destroying or de-identifying personal information that is no longer needed [21].

7. Key Workflows

Registration and sign-in. Self-registration always creates a volunteer. A verification email is queued. The volunteer can sign in straight away, but cannot apply to an event until the address is confirmed. The first administrator is created on first start from the `ADMIN_*` environment variables.

Account administration. Administrators can invite a user with a chosen role, change roles, disable or enable an account, mark an email as verified, reset a password, resend an invitation and end all sessions. A volunteer cannot be promoted while they have a submitted or accepted application, or an upcoming shift. A coordinator who is demoted to volunteer is removed as co-manager from every event. Every role change is kept in `role_change` for history.

Event management. A coordinator creates an event as a draft, adds roles and shifts, then publishes it. The owner adds co-managers by exact email address, and only an existing coordinator account can be added. Only an administrator can invite someone without an account as a new coordinator. Cancelling an event notifies everyone allocated to it.

Form builder. Each event has one application form made of typed questions (short text, long text, single choice, multiple choice, dropdown, checkbox and consent). Questions are never edited in place. Changing a question writes a new version with the same `question_key`, and removing one sets `retired_at`. Answers point at the exact version that was answered, so older applications still read correctly after the form changes. A published form can be shared through a link that contains a random share token.

Applications and review. Before accepting an application the server checks the event's eligibility rules against the volunteer's profile: age for adults-only events, declared checks such as a Working with Children Check, expertise areas and a degree where required. The frontend shows the same rules as a checklist, so a volunteer knows what is missing before applying. Reviewers accept or reject with an optional reason, and the volunteer is told by email and in their application list.

Shift allocation. Accepted volunteers are placed on shifts. The service takes a row lock on the shift while allocating, so two coordinators cannot fill the last place at the same moment (`409 SHIFT_FULL`). If the volunteer already has a shift at an overlapping time, the request is refused with `409 OVERLAPPING_SHIFT` unless the coordinator confirms it, and the confirmation is recorded on the allocation.

Attendance. Managers record each allocated volunteer as present, late or absent, and can correct a record later. The correction keeps who changed it and when. Present and late count towards the volunteer's hours.

Email delivery. Services never send email directly. They add a row to the `email_message` outbox in the same transaction as the change that caused it [22]. A scheduled dispatcher sends queued messages in batches and retries failures with a growing delay (one minute, doubling each attempt). After five attempts the message is marked abandoned. A mail server outage therefore never fails or rolls back the user's action.

Platform settings. Retention, mail delivery, sign-in and password reset limits, the privacy notice version and link, and the temporary password lifetime are stored in a single `platform_settings` row. On first start it is filled from the environment variables. Afterwards administrators change it on the Settings

screen. The scheduled jobs read the current values before each run, so a change takes effect without a restart.

8. Frontend Implementation Notes

Session handling. On start the client requests `/auth/csrf` to obtain the CSRF cookie, then calls `/auth/me` to find out who is signed in. `SessionProvider` holds the result. If any later request returns `401`, the client clears its cache and returns to the sign-in page. If a request returns `PASSWORD_CHANGE_REQUIRED`, it sends the user to the set-password screen. After sign-in the user is returned to the page they asked for, but only if that path is one their role can open.

Data fetching. Each feature defines its query keys next to its API calls. Cached data is treated as fresh for 30 seconds and refetched when the window regains focus. Requests that fail with an error that will not change on retry (`UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND`, `VALIDATION_FAILED`) are not retried, and changes are never retried automatically. After a successful change the related queries are invalidated.

Forms and validation. The sign-in, account, profile and event forms use React Hook Form with a Zod schema. Smaller forms check their few fields directly. Field errors returned by the server are mapped back onto the matching form fields, so server-side validation messages appear next to the right input.

UI and accessibility. Shared primitives in `shared/ui` (buttons, fields, dialogs, menus, pagination, status pills and loading, empty and error states) keep screens consistent. Colours are defined as design tokens and each text and status colour was checked against WCAG 2.2 AA contrast [23]. Every interactive element has the same visible focus outline, dialogs use the native `<dialog>` element so focus is trapped and Escape closes them, and layouts are tested at phone and desktop widths.

Dates and times. The API always sends UTC instants. The frontend converts them for display with `@internationalized/date` in `shared/time`, using the Australia/Adelaide time zone. An ESLint rule blocks the `Date` global outside that module and the tests, since a `Date` built from a string uses the browser's own time zone.

9. Quality Assurance

Layer	Tooling	Scope
Backend tests	JUnit 6, Spring Boot Test, Spring Security Test	272 tests over services, access policies, controllers and repositories
Database tests	Same, tagged `database`	Behaviour that exists only in PostgreSQL, such as the audit triggers. Excluded by default; included with `-Dtest.excludedGroups=`
Frontend unit tests	Vitest, Testing Library, jsdom	93 tests over components, schemas, eligibility logic, the API client and time helpers
End-to-end tests	Playwright	70 tests in 21 spec files covering each role's main journeys, plus accessibility, contrast and responsive layout checks
Static checks	ESLint, Prettier, `tsc`	Lint, formatting and type checking on all frontend code

The end-to-end suite seeds its own accounts and events in a global setup step through the public API, so it does not depend on data left by an earlier run.

10. Local Development Environment

A local environment has four parts:

Service	How it runs	Address
PostgreSQL 17	`docker compose up -d` in `backend/`	localhost:5432
Mailpit	Same compose file	SMTP on 1025, web inbox on http://localhost:8025
Backend API	`./mvnw spring-boot:run` in `backend/`	http://localhost:8080/api
Frontend	`npm run dev` in `frontend/`	http://localhost:5173

The backend reads its configuration from environment variables, with a git-ignored `.env` file as a fallback (Appendix B). The frontend needs a single variable, `VITE_API_BASE_URL`, pointing at the API. Because the session cookie is `Secure` by default, `SESSION_COOKIE_SECURE=false` must be set when running over plain HTTP locally.

Configuration falls into two groups. Connection details, CORS origins and the first administrator are read from the environment at start-up and need a restart to change. Operational limits such as retention, mail retries and sign-in throttling are seeded from the environment once and then managed in the database through the Settings screen (Section 7).

The health of a running backend can be checked at `/api/actuator/health`.

11. Known Limitations

- The schema is managed by Hibernate's ddl-auto=update rather than versioned migrations such as Flyway. This is fine for adding tables and columns, but renames and removals need manual SQL.
- The scheduled jobs assume a single backend instance. Running several instances would need a shared lock so that emails and retention runs are not processed twice.
- Emails containing a temporary password stay in the outbox table, in plain text, until the account is de-identified.
- Degree documents are stored in the database as binary data, which increases database size and backup time as usage grows.

12. References

[1] Oracle, "Oracle Java SE Support Roadmap." Accessed: Oct. 26, 2026. [Online]. Available: https://www.oracle.com/java/technologies/java-se-support-roadmap.html

[2] Broadcom, "Spring Boot Reference Documentation." Accessed: Oct. 26, 2026. [Online]. Available: https://docs.spring.io/spring-boot/

[3] Broadcom, "Spring Security Reference." Accessed: Oct. 26, 2026. [Online]. Available: https://docs.spring.io/spring-security/reference/

[4] Broadcom, "Spring Session Reference." Accessed: Oct. 26, 2026. [Online]. Available: https://docs.spring.io/spring-session/reference/

[5] Broadcom, "Spring Data JPA Reference." Accessed: Oct. 26, 2026. [Online]. Available: https://docs.spring.io/spring-data/jpa/reference/

[6] The PostgreSQL Global Development Group, "PostgreSQL 17 Documentation." Accessed: Oct. 26, 2026. [Online]. Available: https://www.postgresql.org/docs/17/

[7] springdoc, "springdoc-openapi." Accessed: Oct. 26, 2026. [Online]. Available: https://springdoc.org/

[8] OpenAPI Initiative, "OpenAPI Specification v3.1.0," 2021. [Online]. Available: https://spec.openapis.org/oas/v3.1.0

[9] Meta Open Source, "React." Accessed: Oct. 26, 2026. [Online]. Available: https://react.dev/

[10] Vite, "Getting Started." Accessed: Oct. 26, 2026. [Online]. Available: https://vite.dev/guide/

[11] TanStack, "TanStack Query Overview." Accessed: Oct. 26, 2026. [Online]. Available: https://tanstack.com/query/latest/docs/framework/react/overview

[12] React Hook Form, "Get Started." Accessed: Oct. 26, 2026. [Online]. Available: https://react-hook-form.com/get-started

[13] Zod, "Introduction." Accessed: Oct. 26, 2026. [Online]. Available: https://zod.dev/

[14] Tailwind Labs, "Tailwind CSS Documentation." Accessed: Oct. 26, 2026. [Online]. Available: https://tailwindcss.com/docs

[15] openapi-typescript, "Introduction." Accessed: Oct. 26, 2026. [Online]. Available: https://openapi-ts.dev/

[16] Mailpit, "Mailpit Documentation." Accessed: Oct. 26, 2026. [Online]. Available: https://mailpit.axllent.org/

[17] Microsoft, "Playwright." Accessed: Oct. 26, 2026. [Online]. Available: https://playwright.dev/

[18] OWASP Foundation, "Password Storage Cheat Sheet." Accessed: Oct. 26, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[19] OWASP Foundation, "Session Management Cheat Sheet." Accessed: Oct. 26, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[20] OWASP Foundation, "Cross-Site Request Forgery Prevention Cheat Sheet." Accessed: Oct. 26, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

[21] Office of the Australian Information Commissioner, "Australian Privacy Principles." Accessed: Oct. 26, 2026. [Online]. Available: https://www.oaic.gov.au/privacy/australian-privacy-principles

[22] C. Richardson, "Pattern: Transactional outbox," microservices.io. Accessed: Oct. 26, 2026. [Online]. Available: https://microservices.io/patterns/data/transactional-outbox.html

[23] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," 2023. [Online]. Available: https://www.w3.org/TR/WCAG22/

Appendix A. Endpoint Catalogue

All paths are relative to `/api/v1`, except the public `GET /api/utils/isLive` check. "Staff" means a coordinator who owns or co-manages the event, or any administrator.

Method	Path	Purpose	Access
GET	/auth/csrf	Issue the CSRF cookie	Public
POST	/auth/register	Create a volunteer account	Public
POST	/auth/login	Sign in	Public
POST	/auth/logout	Sign out	Signed in
GET	/auth/me	Current account	Signed in
POST	/auth/verify-email, /auth/verify-email/resend	Confirm email, resend link	Public
POST	/auth/password-reset, /auth/password-reset/confirm	Request and complete a reset	Public
POST	/auth/password	Change own password	Signed in
GET	/users	List and search accounts	Administrator
POST	/users/invitations	Invite a user with a role	Administrator
GET	/users/{id}	Account details	Administrator or the account holder
PATCH	/users/{id}/role, /users/{id}/status	Change role, disable or enable	Administrator
POST	/users/{id}/email-verification	Mark email as verified	Administrator
POST	/users/{id}/sessions/revoke	End all sessions	Administrator
POST	/users/{id}/password-reset	Issue a temporary password	Administrator
POST	/users/{id}/invitation/resend	Resend invitation	Administrator
GET	/users/{id}/role-changes	Role history	Administrator
GET, PUT	/profiles/me	Own profile	Volunteer
PUT	/profiles/me/expertise	Expertise areas	Volunteer
POST, GET, DELETE	/profiles/me/degree	Degree document (PDF)	Volunteer
POST, DELETE	/profiles/me/qualifications, /profiles/me/qualifications/{qualificationId}	Qualifications	Volunteer
GET	/profiles/{userId}	A volunteer's profile	The volunteer or an administrator
GET	/events	List events visible to the caller	Signed in
POST	/events	Create an event	Coordinator, administrator

Method	Path	Purpose	Access
GET, PUT	/events/{id}	Read or update an event	Read: published or staff; update: staff
PATCH	/events/{id}/status	Publish, cancel or complete	Staff
POST, DELETE	/events/{id}/managers, /events/{id}/managers/{coordinatorId}	Add or remove a co-manager	Owner, administrator
GET	/events/{id}/managers/lookup	Find a coordinator by email	Owner, administrator
POST	/events/{id}/managers/invitations	Invite a new coordinator	Administrator
PATCH	/events/{id}/owner	Transfer ownership	Administrator
POST, PUT, DELETE	/events/{id}/roles, /events/{id}/roles/{roleId}	Event roles	Staff
POST	/events/{id}/roles/{roleId}/shifts	Create a shift	Staff
GET	/events/{id}/shifts	List shifts	Anyone who can view the event
PUT, DELETE	/shifts/{id}	Update or remove a shift	Staff
POST, GET	/events/{id}/form	Create or read the form	Staff (volunteers can read)
POST	/events/{id}/form/publish	Publish the form	Staff
POST, PUT, DELETE	/events/{id}/form/questions, /events/{id}/form/questions/{questionKey}	Add, revise or retire a question	Staff
GET	/public/forms/{shareToken}	Published form by share link	Public
POST	/applications	Submit an application	Volunteer
GET	/applications/me	Own applications	Volunteer
GET, POST	/applications/me/decisions/unread, /acknowledge	Unread decisions	Volunteer
GET	/applications/{id}	Application details	Applicant or staff
PATCH	/applications/{id}/withdraw	Withdraw	Applicant
PATCH	/applications/{id}/decision	Accept or reject	Staff
GET	/applications/{id}/volunteer, /volunteer/degree	Applicant details and degree	Staff
GET	/events/{id}/applications	Applications for an event	Staff
POST, GET	/shifts/{id}/allocations	Allocate, list allocations	Staff
DELETE	/allocations/{id}	Remove an allocation	Staff
GET	/allocations/me	Own shifts	Volunteer
PUT	/allocations/{id}/attendance	Record or correct attendance	Staff

Method	Path	Purpose	Access
GET	/shifts/{id}/attendance	Attendance for a shift	Staff
GET	/attendance/me	Own attendance and hours	Volunteer
GET	/audit-events	Activity log	Administrator
GET, PUT	/admin/settings	Platform settings	Administrator
GET	/public/privacy-notice	Privacy notice	Public

Appendix B. Environment Variables

Backend, read at every start

Variable	Purpose	Local default
DB_URL, DB_PORT, DB_NAME	Database host, port and name	localhost, 5432, postgres
DB_USERNAME, DB_PASSWORD	Database credentials	postgres, postgres
MAIL_HOST, MAIL_PORT	SMTP server	localhost, 1025
MAIL_USERNAME, MAIL_PASSWORD	SMTP credentials	empty
MAIL_SMTP_AUTH, MAIL_SMTP_STARTTLS	SMTP authentication and TLS	false, false
MAIL_FROM	Sender address	no-reply@nuvexi.local
CORS_ALLOWED_ORIGINS	Origins allowed to call the API	http://localhost:5173
SESSION_COOKIE_SECURE	Send cookies over HTTPS only	true (set false locally)
CLIENT_BASE_URL	Base of links placed in emails	http://localhost:5173
ADMIN_EMAIL, ADMIN_NAME, ADMIN_PASSWORD	First administrator, created once; skipped when no email is set	empty, Administrator, empty

Backend, used once to seed platform settings

Variable	Purpose	Default
RETENTION_PERIOD	How long volunteer data is kept, at most 12 months	P12M (12 months)
RETENTION_SCAN_INTERVAL, RETENTION_BATCH_SIZE	Retention job schedule and batch size	24h, 100
MAIL_DISPATCH_INTERVAL, MAIL_BATCH_SIZE	Dispatcher schedule and batch size	30s, 50
MAIL_MAX_ATTEMPTS, MAIL_BASE_BACKOFF	Retry limit and first retry delay	5, 1m
TEMPORARY_PASSWORD_TTL	Lifetime of a temporary password	72h
PRIVACY_NOTICE_VERSION, PRIVACY_NOTICE_URL	Current privacy notice	draft-0.1, /privacy

Frontend

Variable	Purpose	Local value
VITE_API_BASE_URL	Base URL of the API	http://localhost:8080/api/v1

Appendix C. Error Codes

Code	HTTP status	Meaning
INVALID_INPUT	400	The request contained invalid data
VALIDATION_FAILED	400	One or more fields failed validation; see field errors
MALFORMED_REQUEST	400	The body could not be read
UNAUTHENTICATED	401	Not signed in, or the session has ended
FORBIDDEN	403	The role or record does not allow this action
PASSWORD_CHANGE_REQUIRED	403	A temporary password must be replaced first
NOT_FOUND	404	The record does not exist or is not visible to the caller
CONFLICT	409	The request conflicts with the record's current state
STALE_RESOURCE	409	Someone else changed the record; reload and retry
OVERLAPPING_SHIFT	409	The volunteer already has a shift at that time
SHIFT_FULL	409	The shift has no free places
RATE_LIMITED	429	Too many attempts; try again later
INTERNAL_ERROR	500	Unexpected server error